기초 파이썬

2주차: 2026. 9. 9일

담당교수: 박진호



서강대학교 경제대학원

2026년 2학기

3장: 데이터 전처리 - Garbage In, Garbage Out

◆ [데이터 전처리]는 데이터 과학자 또는 계량경제학자가 업무 시간의 80%를 보낸다는 말이 있을 정도로 중요하며, 실제 책자와 강의에서도 실습 비중이 가장 큰 부분

■ 학습 목표:

(1) 원천 데이터(Raw Data)를 분석 가능한 'Tidy Data'로 변환한다.

(2) 결측값과 중복값 등 데이터 품질 저해 요소를 식별하고 처리한다.

(3) 다양한 형태의 데이터를 병합하고 재구조화하여 분석의 기반을 닦는다.

제3장 데이터 전처리 (1)

3.1 입력과 출력

3.2 자료형(Data Type)

3.3 연산(Operation)

3.4. 숫자

3.5 자료형의 변환

3.6 조건문

3.7 반복문

3.8 함수

3.9 모듈

3.10 클래스

3.11 구문 오류와 예외 처리

3.12 파일 입·출력

3.1 입력과 출력

3.1.1 변수(Variables)

◇ **변수**는 데이터를 저장하는 공간이자, 값을 가리키는 식별자(identifier)

- o **엑셀 비유**: 엑셀의 '셀(cell)'과 같음
  - ▸ 셀 H12에 3.0이 들어있다면, =H12라고 쓸 때 엑셀은 'H12'라는 글자가 아닌 그 안의 값인 3.0을 인식
- o **식별자**: 변수, 함수 등에 붙이는 이름

1 변수의 생애 주기

(1) **변수 선언**: 변수를 생성하는 단계

(2) **변수 할당**: = 기호를 사용하여 값을 넣는 단계

- ▸ =은 '같다'가 아니라 '우변의 값을 좌변에 넣겠다'는 의미

```
price = 1000      # price라는 변수에 1000을 할당
quantity = 5      # quantity라는 변수에 5를 할당
```

(3) **변수 참조**: 변수에 저장된 값을 꺼내어 사용하는 단계

```
print("revenue =", price * quantity)        # 결과: revenue = 5000
```

2 변수 명명 규칙

구분	규칙 내용	예시
시작 문자	▸ 문자(alphabet) 또는 언더바(_)로만 시작 가능 ▸ **숫자**로 시작되는 변수 이름은 불가	_alpha (O), 2025a (X)
사용 가능 문자	알파벳, 숫자, 언더바(_)	gdp_2025 (O), gdp-2025 (X)
대소문자 구분 (case-sensitive)	Alphabet과 alphabet은 서로 다른 변수	Value ≠ value
공백 불가	이름 사이에 띄어쓰기 불가능	GDP_US (O), GDP US (X)
키워드 금지	파이썬 예약어는 사용 불가	if, for, return 등 사용 불가

o 키워드(keyword): 파이썬이 만들어질 때 사용하겠다고 예약한 단어로 **특별한 의미**가 부여되어서 프로그래밍 언어에서 사용이 금지

▸ 예: return, try, for, while, with, yield, False, None, True, and, or, as, assert, lambda, not, break, pass, raise, if, elif, else, finally, import, in, is, del, def, async, await, except, from, global

▸ 예약어 확인 방법:

```
import keyword
print(keyword.kwlist)
```

3 표기법(case style)

□ 긴 변수 이름을 만들 때, 가독성을 위해 사용하는 방식

① **snake_case (스네이크 케이스) 방식:** 단어 사이에 _ 로 연결

- 예: korea_exchange_rate
- 용도: 파이썬에서 **변수**나 **함수 이름**을 지을 때 주로 사용

② **CamelCase (캐멀 케이스) 방식:** 단어의 첫 글자를 대문자로 연결

- 예: KoreaExchangeRate
- 용도: 파이썬에서 **클래스(class) 이름**을 지을 때 사용

3.1.2 print( ) 함수

◇ 데이터를 화면에 출력하는 함수

1 문자열 출력

o print( ) 함수의 괄호 내에 큰 따옴표(" ")나 작은 따옴표(' ') 모두 사용 가능

print("Hello World"), print('Hello World')

o 따옴표 3개(""" """ 또는 ''' ''')를 사용하면 줄바꿈과 띄어쓰기가 그대로 유지됨

```
print("""AI technology mimics
human intelligence.""")
```

2 특수문자 처리(escape character)

o **\'** : 문자열 안에서 따옴표를 문자로 인식하게 함 (print("Let\'s go together"))

o **\n** : 줄바꿈 (new line) (print("Hello World**\n**Let's go together"))

o **\t** : 탭(tab) 공백

3 주요 옵션 (sep, end)

o **sep** (separator): 쉼표로 구분된 값들 사이의 문자를 지정 (기본값: 공백)

```
print('2025', '03', '13', sep='-')          # 결과: 2025-03-13
print('user', 'gmail.com', sep='@')    # 결과: user@gmail.com
```

o **end**: 출력이 끝난 후 뒤에 붙을 문자를 지정 (기본값: \n)

```
print("Hello", end=" ")
print("World")                              # 결과: Hello World (한 줄에 출력)
```

4 변수 및 자료형 확인

o 여러 변수를 동시에 선언하거나, 사칙연산을 수행하여 출력할 수 있음

▸ 단, 사칙연산은 정수형 간, 실수형 간에 가능

```
a = 10
b, c = 20, 30
```

```
d = b + c
print("a =", a)
print("d =", d)
```

o type() 함수를 함께 사용하면, 데이터의 성격(문자열, 숫자 등)을 알 수 있음

```
country = "Korea"
phi = 3.141592653
print(type(country))    # <class 'str'>
print(type(phi))        # <class 'float'>
```

3.1.3 주석(comment)

□ ① 코드에 설명을 달거나, ② 특정 코드가 실행되지 않도록 할 때 사용

o 기호: #을 사용하며, 그 뒤의 내용은 프로그램 실행시 무시됨

```
# 이것은 가격을 설정하는 코드입니다.
price = 1000        # 코드 뒤에도 작성 가능
```

3.1.4 들여쓰기

□ 파이썬은 R에서 사용하는 **중괄호 { }** 대신 **공백(tab/space)**으로 코드의 종속관계를 구조화함

o 규칙: **콜론(:)**으로 끝나는 문장 다음 줄은 반드시 들여쓰기를 해야 함

o 표준: 일반적으로는 스페이스 4칸을 권장

```
scores = [85, 62, 90, 55, 73]

for score in scores:
    if score >= 70:
        print(score, "합격")      # if에 속한 코드
    else:
        print(score, "불합격")    # else에 속한 코드
```

3.1.4 help( ) 함수

□ 함수나 클래스의 자세한 사용법(도움말)을 확인하고 싶을 때 사용

```
help("print")          # print 함수의 옵션과 설명이 출력됨
```

3.2 데이터 유형(Data Type)

3.2.1 데이터, 데이터의 형식, 문자열, print( ) 함수

◇ 데이터는 프로그램이 처리하는 모든 요소를 의미하며, 그 성격에 따라 데이터 유형(data type)을 분류할 수 있음

1 데이터의 형식 확인: type( ) 함수

□ <class 'str'>: 문자열(string). 따옴표로 감싸진 글자들의 나열

□ <class 'int'>: 정수(integer). 소수점이 없는 숫자

```
print(type("Hello World"))      # 결과: <class 'str'>
print(type(2025))               # 결과: <class 'int'>
```

2 문자열(string) 만들기

□ 문자열은 글자들을 나열한 것으로, 반드시 큰 따옴표(“) 또는 작은 따옴표(’)로 문자 앞뒤에서 감싸야 함

□ 성공 사례 (서로 다른 따옴표 사용):

- "큰따옴표 안에 '작은따옴표' 넣기" (O)
- '작은따옴표 안에 "큰따옴표" 넣기' (O)

□ 실패 사례 (동일한 따옴표 사용):

- "Powell said "net effect" of changes..." (X) → 중간의 "에서 문자열이 끝난 것으로 인식

3 이스케이프 문자(escape character)

□ 이스케이프 문자는 역슬래시(\ 또는 ₩)와 문자를 조합하여 특수한 기능을 수행하게 하는 문자

(1) 따옴표 자체를 문자로 인식시키기

o 동일한 종류의 따옴표를 내부에서 써야만 할 때, 역슬래시(\)를 앞에 붙이면 기호가 아닌 '문자 그 자체'로 인식됨

▸ back slash(＼) = \ (한국어 키보드). forward slash(／)

```
print("Powell added that the \"net effect\" of those changes...")
print('It\'s a beautiful day.')
```

(2) 줄바꿈과 탭 활용

이스케이프 문자	기능	활용 예시
\n	줄바꿈	print(Hello\nWorld)
\t	탭(Tab)	데이터 간격을 맞추거나 표 형식을 만들 때 사용

o 탭(**\t**)을 이용한 표 형식 출력 예시:

```
print("국가 \t 기업명 \t 업종")
print("한국 \t 현대자동차 \t 제조업")
```

4 여러 줄 문자열 출력

o 내용이 아주 길거나 실제 작성한 모양 그대로 줄바꿈을 적용하고 싶을 때는 따옴표 3개를 사용함

- 기호: """ """ 또는 ''' '''
- 특징: 엔터(Enter)를 치는 곳마다 자동으로 줄바꿈이 적용되어 출력됨

```
# 긴 연설문이나 기사를 출력할 때 유용
print("""Washington(CNN) –
America is feeling uneasy about the Trump administration's rapid-fire trade policy,
but Federal Reserve Chair Jerome Powell said Friday he's not worried.""")
```

◆ (정리)

1. 구문 오류(Syntax Error): 문법 규칙을 어겼을 때 발생(예: 괄호 안 닫음)

2. 역슬래시 인식: 한국어 키보드에서는 ₩ 버튼이 역슬래시(\) 역할

3. 가독성: 아주 긴 문장은 \n을 여러 번 쓰는 것보다 """를 사용하는 것이 코드를 읽기에 더 편함

3.2.2 기본 자료형

◆ 변수가 저장되는 데이터의 형태를 뜻하며, 파이썬의 가장 기본적인 유형은 다음과 같음

자료형	영문명 (Type)	특징	예시
정수형	int (integer)	소수점이 없는 숫자	0, 1, -5
실수형	float (floating-point)	소수점이 포함된 숫자	3.14, 2.0, 0.0
문자형	str (string)	따옴표로 감싸진 글자나 숫자	“Hello”, “2025”
불린형	bool (boolean)	참(True) 또는 거짓(False)	True (1), False (0)

3.3 연산(Operation)

3.3.1 문자열 연산(string operation)

1 연결(+)과 반복(*)

o 연결(+): 문자열과 문자열을 하나로 합침. “문자열” + “문자열”

“Hello” + “World” ➜ HelloWorld

o 주의: 문자열과 숫자는 직접 더할 수 없음

▸ 숫자를 " "로 감싸 문자열로 만들어야 함 (“Hello World” + “2025”)

o 반복(*): 문자열을 지정한 횟수만큼 반복 ("Hello World "*3)

2 인덱싱(indexing)과 슬라이싱(Slicing)

o 파이썬은 숫자를 0부터 세는 제로 인덱스(zero-based indexing)를 사용

* 숫자를 1부터 세는 **원 인덱스(one index)**는 대표적으로 R

o **인덱싱 [ ]**: 특정 위치의 문자 하나를 선택

▸ "Hello"[0] → H # 0은 "첫 번째 원소로부터 얼마나 떨어져 있는가"라는 개념

▸ "Hello"[-1] → o (뒤에서 첫 번째)

H	e	l	l	o		W	o	r	l	d
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]

```
print("인덱싱")
print("Hello World"[0])
print("Hello World"[1])
⋮
print("Hello World"[10])
```

▸ 대괄호 [ ] 안의 숫자를 음수(-)로 입력하면 뒤로부터 선택

H	e	l	l	o		W	o	r	l	d
[-11]	[-10]	[-9]	[-8]	[-7]	[-6]	[-5]	[-4]	[-3]	[-2]	[-1]

```
print("음수로 인덱싱")
print("Hello World"[-1])
⋮
print("Hello World"[-11])
```

o **슬라이싱 [시작:끝]:** 범위로 부분 문자열을 추출

- **대괄호 안 [ ]에 있는** 문자열의 범위를 콜론(:)으로 구분하여 지정해 문자열 일부를 추출하는 것을 슬라이싱(slicing)이라 함

o 파이썬은 지정된 범위의 명령을 수행할 때 마지막 숫자(끝 번호)포함하지 않고 실행

- 지정된 범위에 있는 (마지막 숫자 - 1)까지 선택
- **a[start:stop]**에서 start는 포함하고, stop은 포함하지 않는다.

H	e	l	l	o		W	o	r	l	d
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]

```
print("Hello World"[1:7])
ello W

print("Hello World"[0:5])
Hello
```

o 대괄호 [ ] 안에 넣는 숫자에서 **좌우 하나를 생략**

① **우측 숫자**를 미지정 → 마지막 글자까지 지정하라는 뜻

② **좌측 숫자**를 미지정 → 첫 번째 글자부터 지정하라는 뜻

```
print("Hello World"[2:])
llo World

print("Hello World"[:7])
Hello W
```

□ IndexError 예외

o IndexError 예외는 문자열/리스트의 수를 넘는 글자/요소를 선택할 때 발생하는 것으로 프로그래밍에서 가장 많이 발생하는 예외 중 하나

– IndexError: index out of range

```
print("Hello World"[12])
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[13], line 1
----> 1 print("Hello World"[12])

IndexError: string index out of range
```

□ **len( ) 함수**: 괄호 내부에 문자열을 넣으면 공백을 포함한 전체 문자의 개수를 반환

```
print(len("Hello World"))
 11
```

3.4. 숫자

3.4.1 숫자의 종류

□ 정수(integer)는 소수점이 없는 숫자, 실수(floating point, 부동소수점)는 소수점이 있는 숫자를 각각 의미

o 정수: 0, 1, 5, 10, -22, -76

o 실수: 0.0, 1.5, 3.141529, -22.0005

o 0은 (소수점이 없는) 정수, 0.0은 (소수점이 있는) 실수여서 숫자의 자료형이 다름

o 부동 소수점(floating point): 소수점이 움직이는 숫자

– 예: 3.141529 = 0.03141592 × 100 =

$$0.03141592 \times 10^2 = 0.03141592e2 = 0.03141592E2$$

□ 숫자의 유형 확인: type( ) 사용

```
print(type(10))
<class 'int'>
print(type(3.14))
<class 'float'>
```

→ <class 'int'>에서 **int는 정수를**, <class 'float'>에서 **float는 실수를** 의미

```
print(type(0))
<class 'int'>
print(type(0.0))
<class 'float'>
```

□ **숫자의 자료형 구분이 필요한 이유**는 파이썬에서 인덱싱(문자열 선택 연산자) [ ] 등을 사용시 대괄호 [ ] 안에 넣는 숫자가 **정수**가 아니라 **실수**이면 오류 발생 등의 흔치 않은 작은 차이가 있기 때문임

```
print("Hello World"[0])
print("Hello World"[0.0])
```

3.4.2 숫자의 연산자

□ **사칙 연산자: + (덧셈), - (나눗셈), * (곱셈), / (나눗셈)**

```
print("10 + 20 = ", 10 + 20)
```

```
10 + 20 =  30
```

```
print("10 - 20 = ", 10 - 20)
```

```
10 - 20 =  -10
```

```
print("10 * 20 = ", 10 * 20)
```

```
10 * 20 =  200
```

```
print("10 / 20 = ", 10 / 20)
```

```
10 / 20 =  0.5
```

□ 몫과 나머지 연산자: // (몫), % (나머지)

```
print("5 // 3 = ", 5 // 3)
```

```
5 // 3 =  1
```

```
print("5 % 3 = ", 5 % 3)
```

```
5 % 3 =  2
```

□ 제곱 연산자: ** (제곱, power)

```
print("5 ** 3 = ", 5 ** 3)
```

```
5 ** 3 =  125
```

◈ 주의: Python에서 ^는 제곱 연산자가 아님

```
print("5 ^ 3  =", 5 ^ 3)      # Python에서 ^는 비트 단위 배타적 OR(bitwise XOR) 연산자
```

```
5 ^ 3  = 6                    # 두 정수를 2진수로 바꾼 뒤, 각 자리의 비트(bit)를 서로 비교
```

3.4.2 복합 대입 연산자: 증가 연산, 감소 연산 등

□ 파이썬에서 증가 연산은 **+=**, 감소 연산은 **-=** 로 표시

o 사례: **a = 1**로 선언 후에 **a = a + 1**을 입력하면, 식의 오른쪽에 있는 a는 1로 치환되어 1 + 1의 값이 다시 a에 할당된다는 뜻

```
print("증가 연산")
a = 1                   # 변수 a에 1을 할당
a = a + 1               # 변수 a에 1을 더한 후에 그 값을 다시 a에 할당
print(a)

a = 1                   # 변수 a에 1을 할당
a += 1                  # a의 증가 연산
print(a)
```

```
print("감소 연산")
a = 2                # 변수 a에 2를 할당
a = a - 1            # 변수 a에 1을 뺀 후에 그 값을 다시 a에 할당
print(a)

a = 2                # 변수 a에 2를 할당
a -= 1               # a의 감소 연산
print(a)
```

o 위에서 **a += 1**은 **a = a + 1**과 **같은 의미**

□ **곱셈 연산은 *=, 나눗셈 연산은 /=** 로 표시

```
print("곱셈 연산")
a = 1                # 변수 a에 1을 할당
a = a * 2            # 변수 a에 2를 곱한 후에 그 값을 다시 a에 할당
print(a)
```

```
a = 1                # 변수 a에 1을 할당
a *= 2               # a의 곱 연산
print(a)

print("나눗셈 연산")
a = 1                # 변수 a에 1을 할당
a = a / 2            # 변수 a를 2로 나누어 그 값을 다시 a에 할당
print(a)

a = 1                # 변수 a에 1을 할당
a /= 2               # a의 나눗셈 연산
print(a)
```

□ Q) **=, //=, %=

```
a = 7
a //= 2          # a %= 2
print(a)
```

Q) 아래에서 k의 최종값을 얼마인가?

```
k = 10
k += 10
k -= 5
k /= 5
k **= 2
print("k:", k)
```

□ 문자열 복합 대입 연산자: 문자열도 사용 가능

Q) 아래에서 SyntaxError: 가 발생하는 이유는? 코드를 수정해 보세요.

```
string = "Hello World"
string += !
string += ?
print("string:", string)
```

3.4.3 연산자의 우선순위(Operator Precedence)

□ 숫자를 계산할 때, 곱셈(*)과 나눗셈(/)이 덧셈(+)과 뺄셈(-)보다 우선함

```
print(10 + 5 * 5)
```

□ 그러나, 괄호 ( )로 묶은 숫자는 괄호로 묶지 않은 숫자보다 우선하여 계산

```
print( (10 + 5) * 5 )
```

□ 곱셈. 나눗셈, 덧셈, 뺄셈과같이 동일한 우선순위를 가진 연산자는 왼쪽에서 오른쪽 순서로 계산

```
print("연산자의 우선선위")
print(3 + 3 - 3 * 3 / 3 * 3)

print("계산과정")
print("a1 = 3*3  =",    3 * 3)
print("a2 = a1/3 =",    a1 /3)
print("a3 = a2*3 =",    a2 * 3)
print("a4 = 3+3  =",    3 + 3)
print("a5 = a4 - a3 =", a4 - a3)
```

3.5 자료형의 변환

□ 프로그래밍에서는 어떤 자료형(예: 문자열)을 다른 자료형(예: 숫자)으로 변환하는 사례가 빈번히 발생

o 동일한 자료형을 고려하지 않고 상이한 자료형을 연산하면 TypeError라는 예외가 발생

o 앞에서 문자열은 문자열끼리, 숫자는 숫자끼리 연산해야 연산에서 문제가 발생하지 않는다는 것을 학습

<문자열과 숫자는 더할 수 없기 때문에 오류가 발생>

```
string = "Hello World"
print("문자 + 숫자:", string + 30)
```

⇒ 이러한 오류를 해결하기 위해서는 문자열을 숫자로 변환하거나 숫자를 문자열로 변환하는 것이 필요

3.5.1 문자열을 숫자로 변환

□ 숫자 연산을 실행하기 위해 문자열을 숫자로 변환하는 함수

o int( ) 함수: 문자열을 int 자료형으로 변환. int는 정수형을 나타냄

o float( ) 함수: 문자열을 float 자료형으로 변환. float는 실수형(부동 소수형)을 나타냄

□ 예제: 문자열 연산과 숫자 연산

```
string_a = "1500"
string_b = "1000"
print("문자열 덧셈:", string_a + string_b)
print("--------------------------------")
int_a = int(string_a)
int_b = int(string_b)
print("숫자 덧셈:", int_a + int_b)
```

문자열 덧셈: 15001000

숫자 덧셈: 2500

3.5.2 숫자를 문자열로 변환

□ **str( ) 함수**를 사용하여 숫자를 문자열로 변환

```
string_a = str(15)
string_b = str(15.123)
print(type(string_a), string_a)
print(type(string_b), string_b)
```

```
<class 'str'> 15
<class 'str'> 15.123
```

3.5.3 ValueError 예외

□ 변환될 수 없는 자료형을 아래처럼 변환하려 할 때 ValueError가 발생

(1) 숫자가 아닌 것을 숫자로 변환 시도

o int( ) 함수와 float( ) 함수의 괄호 안에 변환할 수 없는 형태(매개변수)가 들어가는 경우

```
int("Hello World")
float("Hello World")
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[33], line 2
      1 # ValueError 예외: 숫자가 아닌 것을 숫자로 변환할 때
----> 2 int("Hello World")
      3 float("Hello World")
ValueError: invalid literal for int() with base 10: 'Hello World'
```

(2) 소수점이 있는 숫자 형식의 문자열을 int( ) 함수로 변환 시도

o 문자형을 int( ) 함수를 사용해 정수형으로 변환할 때는 오류 발생

```
int("15.123")
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
 Cell In[35], line 1
      ----> 1 int("15.123")

 ValueError: invalid literal for int() with base 10: '15.123'
```

3.5.4 정수형과 실수형 간 자료형 변환

□ int( ) 함수 또는 float( ) 함수를 사용

```
a = 5
print(a)

a = float(5)
print(a)
```

```
5
5.0
```

3.6 조건문

3.6.1 비교 연산자

□ **불리언(Boolean) 또는 불(Bool)**은 True(참)와 False(거짓) 값만을 갖는 자료형으로서 비교 연산자를 사용해 생성

o 기본적인 연산자:

연산자	의미	연산자	의미
==	같다	>	크다
!=	다르다(아니다)	<=	적거나 같다
<	적다	>=	크거나 같다

주: **주어가 왼쪽에 있음**

o 비교 연산자는 숫자 또는 **문자열**에 적용 가능 → Boolean 자료형 생성

① 숫자의 비교

```
print(1 == 2)
print(1 != 2)
print(1 < 2)
print(1 > 2)
print(1 <= 2)
print(1 >= 2)
```

```
False
True
True
False
True
False
```

② 문자열의 비교: 사전 순으로 크기가 정해짐

– 한글: 가나다 순으로 먼저 나오는 문자가 작은 값을 가짐

```
print("교역" == "성장")
```

```
print("교역" != "성장")
print("교역" < "성장")
print("교역" > "성장")
print("교역" <= "성장")
print("교역" >= "성장")
```

```
False
True
True
False
True
False
```

- 영어: abc 순

```
print("abc" == "def")
print("abc" != "def")
print("abc" < "def")
print("abc" > "def")
print("abc" <= "def")
print("abc" >= "def")
```

3.6.2 논리 연산자

□ Boolean 자료형끼리는 논리 연산자를 사용 가능

o 파이썬의 논리 연산자: **not, and, or**

□ **not 연산자**

o 참과 거짓을 반대로 전환할 때 사용

```
print(True)
print(not True)
print(False)
print(not False)
```

```
True
False
False
True
```

```
a = 5
less_10 = a < 10
print("less_10:", less_10)
print("not less_10:", not less_10)
```

□ **and 연산자: Logical AND ➜** and는 두 값의 논리 AND를 반환

o and는 **두 항목의 값이 모두 참일 때만 True**를 반환

```
# and (Logical AND)
print(True and True)
print(True and False)
print(False and True)
print(False and False)
```

```
True
False
False
False
```

```
a = 5
print(0 < a)
print(a < 10)
print(0 < a and a < 10)
```

```
True
True
True
```

□ **or 연산자: Logical OR ➜** or은 두 값의 논리 OR을 반환

o **or**은 두 항목 중 **하나만 참이어도 True**를 반환

```
print(True or True)
print(True or False)
print(False or True)
print(False or False)
```

```
True
True
True
False
```

Truth Table for Logical Operators in Python

X	Y	X and Y	X or Y	not(X)	not(Y)
T	T	T	T	F	F
T	F	F	T	F	T
F	T	F	T	T	F
F	F	F	F	T	T

주: T = True, F = False

3.6.3 if 조건문

□ **if 조건문**은 **조건에 따라** 코드를 실행하거나, 실행하지 않게 만드는 구문

o **if 조건문의 뒤**에는 반드시 **콜론(:)**을 붙여야 함

① if 뒤에 오는 값(Boolean 값)이 참이므로, 들여쓰기(네 번 뛰어쓰기)된 명령문을 실행

```
if True: Enter
    print("True") Enter
```

② if 뒤에 오는 값(Boolean 값)이 거짓이므로. 들여쓰기(네 번 뛰어쓰기)된 명령문을 실행

```
if False: Enter
    print("False") Enter
```

```
# if statement & Boolean
Integer = True
if Integer:
    print("Number is integer.")
```

```
Number is integer.
```

```
# if statement & Boolean
Integer = 10
if Integer > 0:
    print("양수")
```

3.6.4 else 조건문

□ if 조건문은 여러 경우가 있을 때 일일이 적용한다면 비효율을 초래

```
# if 조건문
number = int(5.0)
if number % 2 == 0:
    print("짝수")
if number % 2 != 0:
    print("홀수")
```

□ **else 조건문**은 if 조건문 뒤에 사용하며, **if 조건문의 조건이 거짓이면 실행**

```
if 조건:
■ ■ ■ ■ if 조건이 참이면 실행될 문장
else:
■ ■ ■ ■ if 조건이 거짓이면 실행될 문장
```

```
# if statement & Boolean
Integer = False
if Integer:
    print("Number is integer.")
else:
    print("Number is not integer.")



# if else 조건문: 홀짝수 구분
number = int(5.0)
if number % 2 == 0:
    print("number는 짝수")
else:
    print("number는 홀수")
```

3.6.5 elif 구문

□ **세 개 이상의 조건**을 연결해서 사용할 때 적용하는 방법

```
if 조건 A:
    조건 A가 참이면 실행되는 문장
elif 조건 B:
    조건 B가 참이면 실행되는 문장
elif 조건 C:
    조건 C가 참이면 실행되는 문장
 ...
else:
    앞의 모든 조건이 거짓이면 실행되는 문장
```

```
# elif 구문
 x = 7
 if x % 2 == 0:
      print("x is divisible by 2")
 elif x % 3 == 0:
      print("x is divisible by 3")
 else:
      print("z is neither divisible by 2 nor by 3")
```

3.7 반복문

3.7.1 리스트(list)

◇ 리스트는 **숫자, 문자열, 불린 등 여러 형태의 자료**를 한데 모아 저장할 수 있는 자료형

1 리스트 생성과 접근

□ **선언**: 대괄호 [ ] 안에 요소(element)를 쉼표(,)로 구분해 입력

□ **인덱스(index)**:

- **자료의 위치를 나타내는 숫자**로, 0부터 시작. 대괄호 [ ] 안에 들어간 숫자
- **리스트 추출: 리스트 이름 바로 뒤에** 대괄호 [ ]를 입력한 후 **위치**를 표시하는 숫자를 입력

□ **변경**: 특정 인덱스를 지정해 새로운 값을 할당

```
list = [5, 3.2, 100, "수출", True]

# 요소 접근 및 인덱싱
print(list[0])              # 5
print(list[4])              # True
print(list[-1])             # True (대괄호 [ ] 안에 음수를 넣으면 뒤에서부터 요소 선택)

# 리스트의 범위선택
print(list[1:3])            #

# 리스트 안에 있는 특정 요소를 변경
list[3] = "수입"
print(list)                 #

# 이중 접근 (문자열 인덱싱)
print(list[3][0])           #

# IndexError 예외: 리스트의 길이를 넘은 인덱스로 접근하면 발생하는 예외
list = [5, 3.2, 100, "수출", True]
print(list[9])
```

2 중첩 리스트 (List in List)

□ **리스트 안**에 **리스트**를 포함

```
list_list = [[1, 2, 3], ["수출", "수입", "교역"], [10, 20, 30]]

print(list_list[1])                 #
print(list_list[1][2])               #
```

□ IndexError 예외: 리스트의 길이를 넘은 인덱스로 접근하면 발생하는 예외

```
list = [5, 3.2, 100, "수출", True]
list[9]
```

3 리스트 기본 연산 및 함수

□ **연결(+)**: 두 리스트를 하나로 합침

□ **반복(*)**: 리스트 요소를 n번 반복

□ **길이(len())**: 리스트에 포함된 요소의 개수 확인

```
list_a = [1, 2, 3]
list_b = [7, 8, 9]

print("list_a + list_b =", list_a + list_b)  # list_a + list_b =
print("list_b * 3 =", list_b * 3)            # list_b * 3 =
print("len(list_a) =", len(list_a))          #
```

Q) 중첩 리스트의 길이는?

```
list_list = [[1, 2, 3], ["수출", "수입", "교역"], [10, 20, 30]]
print("len(list_list) =", len(list_list))
```

4 요소 추가: append( ), insert( ), extend( )

함수명	기능	사용법
append(요소)	리스트 맨 뒤에 요소 추가	리스트이름.append(요소)
insert(위치, 요소)	지정한 인덱스 위치에 추가	리스트이름.insert(위치, 요소)
extend([리스트])	여러 요소를 한 번에 추가	리스트이름.extend([100, 200])

```
# append() 함수: 리스트 뒤에 요소 추가
list = [4, 5, 6]
list.append(10)
list.append(11)
print(list)                          #

# insert() 함수: 리스트 중간에 요소 추가
list.insert(0, 99)
print(list)                          #

# extend( ) 함수: 리스트에 여러 요소를 한 번에 추가
# append( ) 함수와 insert( ) 함수: 리스트에 요소 하나를 추가
list = [4, 5, 6]
list.extend([100, 200, 300])
print(list)                    # [4, 5, 6, 100, 200, 300]
```

5 요소 제거

(1) 인덱스로 제거: del 키워드, pop( ) 함수

□ **del 리스트이름[인덱스]**: **특정 위치(인덱스)의 요소**를 **삭제**

- **del**을 먼저 입력하고 리스트 이름과 인덱스를 순차적으로 입력
- **범위를 지정**해 리스트의 요소를 한 번에 제거 가능

□ 리스트.**pop(인덱스)**: **특정 위치의 요소**를 **삭제**

- **인덱스**가 제시되지 않으면 **-1로 인식해 마지막 요소를 제거**

(2) 값으로 제거 및 전체 삭제: remove( ) 함수, clear() 함수

□ 리스트.**remove(값)**: 리스트 내에 일치하는 **특정 값**을 찾아 **삭제**

□ 리스트.**clear()**: 리스트 내부의 **모든 요소를 삭제**하여 빈 리스트([ ])를 만듦

```
# del() 및 pop() 함수: 리스트에서 요소 제거
list = [1, 2, 3, 4, 5, 6, 7]

del list[0]
print("del list[0] =", list)    #

list.pop(3)
print(list)                     # 인덱스 3번 삭제 ->

del list[1:3]                   # [:] 연산자로 범위를 지정해 요소 선택(슬라이싱)
print(list)                     #

del list[:1]                    # 지정된 범위의 왼쪽을 모두 제거
print(list)                     #

del list[1:]                    # 지정된 범위의 오른쪽을 모두 제거
print(list)


# remove ( ) 함수: 리스트의 요소를 값으로 제거
list = [1, 2, 3, 1000, 4, 5]

list.remove(1000)               #
print(list)

list.clear()
print(list)                     #  []
```

⑥ 리스트 정렬 sort() 및 존재 확인 in / not in

□ **sort()** : 데이터를 순서대로 정렬

- ▸ 오름차순(기본): list.sort()
- ▸ 내림차순: list.sort(reverse=True)

□ **in/not in 연산자**: 리스트 내부에 특정 값이 있는지를 확인하여 True/False를 반환

- ▸ in 연산자: 특정 값이 리스트 안에 있는지, not in 연산자: 특정 값이 리스트 안에 없는지

```
# sort(): 정렬
list_a = [10, 5, 3, 222, 6, 1, 9]
list_a.sort()
print(list_a)                   # [1, 3, 5, 6, 9, 10, 222]

list_b = [1, 999, 33, 22, 50, 2, 7]
list_b.sort(reverse=True)
print(list_b)                   # [999, 50, 33, 22, 7, 2, 1]

# in/not in 연산자: 존재 확인
print(222 in list_a)            # True
print(200 not in list_a)        # True
```

3.7.2 for 반복문

◇ 동일하거나 유사한 작업을 여러 번 수행해야 할 때, 코드의 효율성을 높이기 위해 사용

1 **기본 문법:**

for 반복자 in **반복할 수 있는 것(리스트, 문자열, 범위 등)**:
    수행할 코드

① **단순 반복**: range(횟수)를 사용하여 지정된 횟수만큼 반복

```
for i in range(5):
    print("파이썬")          # "파이썬"이 5번 출력됨
```

② 숫자로 된 **리스트와 함께 사용**: 리스트의 각 요소가 **순차적으로** 변수에 대입됨

```
list = [1, 999, 33, 22, 50, 2, 7]

for element in list:
    print(element)
```

③ **문자열과 함께 사용**: 문자열에서 **한 글자씩 꺼내어** 반복

```
for character in "Hello World":
    print("*", character)
```

2 중첩 반복문과 2차원 리스트

□ 반복문 안에 또 다른 반복문을 사용하는 형태. **2차원 리스트(리스트 안의 리스트)**를 처리할 때 유용

▸ 예) 1차원 리스트: [1, 2, 3], 2차원 리스트: [[1, 2, 3], [8, 9, 10]]

```
list_double = [[1, 2], [7, 8, 9], [10, 11, 12, 13]]

# 반복문을 한 번 사용: 덩어리째 출력
for elements in list_double:
    print(elements)                # 내부 요소 개수만큼 '리스트 덩어리'를 반복 출력
```

```
[1, 2]
[7, 8, 9]
[10, 11, 12, 13]
```

```
# 반복문을 두 번 사용: 알맹이만 하나씩 출력 (가장 많이 사용)
 for elements in list_double: # 1번 상자(큰 리스트)를 열어서 덩어리(elements)를 꺼냄
     for item in elements:    # 덩어리 리스트 안에 있는 숫자들을 하나씩 item이라는 이름
                                으로 불러서 처리
         print(elements)      # print(elements). 리스트 내 숫자만큼 순서대로 출력
```

```
[1, 2]
[1, 2]
[7, 8, 9]
[7, 8, 9]
[7, 8, 9]
[10, 11, 12, 13]
[10, 11, 12, 13]
[10, 11, 12, 13]
[10, 11, 12, 13]
```

▸ [1, 2] 안에 알맹이가 2개 있으므로, 덩어리 모양인 [1, 2]를 **두 번 출력**.
그다음 [7, 8, 9] 안에는 알맹이가 3개 있으므로, [7, 8, 9]를 **세 번 출력**.

▸ 그러나, **리스트 안의 숫자들(1, 2, 7, 8, 9...)만 하나씩 쭉 뽑아내고 싶다면:**

```
for elements in list_double:
    for item in elements:
        print(item)  # elements가 아니라 item을 출력!
```

```
1
2
7
8
9
10
11
12
13
```

3 **전개 연산자:** 리스트 앞에 * 기호를 붙여 리스트의 요소를 낱개로 풀어 입력하는 효과를 냄

□ **리스트 합치기:** 리스트 a의 요소들을 나열하여 새 리스트 생성

```
a = [5, 6, 7, 8]
b = [*a, *a]
print(b)                    # [5, 6, 7, 8, 5, 6, 7, 8]
```

□ **리스트에 요소를 추가: append( )와 같은 효과**

```
b = [5, 6, 7, 8]
c = [*b, 100]
print(c)                    # [5, 6, 7, 8, 100]

d = [5, 6, 7, 8]
d.append(100)
print(d)                    # [5, 6, 7, 8, 100]
```

□ **함수 매개변수 위치에 사용**

```
a = [5, 6, 7, 8]
print(*a)                   # 5 6 7 8 (공백으로 구분되어 출력)
```

3.7.3 딕셔너리(dictionary)

◇ 인덱스(숫자) 대신 키(Key)를 기반으로 값을 저장하는 자료구조

1 리스트 vs 딕셔너리

□ **리스트(list)**: **인덱스(index)**를 기반으로 값을 저장

□ **딕셔너리(dictionary)**: **키(key)**를 기반으로 값을 저장

□ **딕셔너리 생성**은 중괄호 { }, **딕셔너리의 요소 접근**은 **대괄호 [ ]** 사용

자료형	위치 지정 방식	생성 형식	예시
리스트	인덱스(Index, 숫자)	**변수 = [ ]**	list[0]
딕셔너리	키(Key, 이름)	**변수 = { }**	dict[“country”]

2 딕셔너리 생성 및 접근

□ **생성**: {**키: 값**} 형태로 작성. key: value

▸ 딕셔너리 안에 들어갈 요소가 여럿이면 **쉼표(,)**로 연결

dictionary = {
"**country**": "독일",
"region": "유럽",
"경제수준": "선진국"
}

▸ **키에는 반드시**(예: country) 따옴표를 붙여야 **변수명**과 혼동되지 않음
(따옴표 없이 입력하면 이를 변수로 인식)

```
# "딕셔너리 생성"
dictionary = {
    country: "독일",
    "region": "유럽",
    "economic_level": "선진국",
}
# "딕셔너리 출력"
print("country:", dictionary["country"])
print("region:", dictionary["region"])
```

KeyError: 'country'

□ **접근**: 딕셔너리 이름 뒤에 대괄호 [ ]를 입력하고 대괄호 [ ] 안에 키를 입력

```
# 딕셔너리 생성. 여기서 dictionary는 임의의 명칭
dictionary = {
    "country": "독일",
    "region": "유럽",
    "economic_level": "선진국",
    "export_item": ["자동차", "기계", "부품"]
}

# 딕셔너리 출력
print("country:", dictionary["country"])
print("region:", dictionary["region"])
print("export_item:", dictionary["export_item"])
```

```
country: 독일
region: 유럽
export_item: ['자동차', '기계', '부품']
```

```
# "값 변경"
dictionary["region"] = "서유럽"
print("region:", dictionary["region"])
```

```
region: 서유럽
```

□ export_item은 dictionary의 **키** 인 동시에 **리스트** 이므로 **인덱스를 지정**하여 **리스트 안의 특정 값을 출력** 가능

```
print(dictionary["export_item"][0])
```

자동차

3 요소 추가/제거

(1) **새로운 자료를 추가**하는 경우 **키와 값을 입력**

```
dictionary["GDP_per_head"] = 45000
print(dictionary)
```

```
{'country': '독일', 'region': '서유럽', 'economic_level': '선진국', 'export_item': ['자동차', '기계', '부품'], 'GDP_per_head': 45000}
```

(2) **딕셔너리 값 대체**: **키를 지정하고 새로운 값을 입력**

```
dictionary["region"] = "서유럽"
```

(3) **딕셔너리 요소의 제거**: **del 딕셔너리[키]** (해당 키와 값을 삭제)

```
del dictionary["region"]
print(dictionary)
```

```
{'country': '독일', 'economic_level': '선진국', 'export_item': ['자동차', '기계', '부품'], 'GDP_per_head': 45000}
```

4 키(Key) 확인 및 오류 방지

□ KeyError: 존재하지 않는 키를 호출할 때 발생

o 리스트: 리스트의 길이를 넘은 인덱스에 접근하면 **IndexError 발생**(index 기반으로 값을 저장하므로)

```
# 딕셔너리 생성
dictionary = { }
# 딕셔너리 키에 접근
dictionary("Bank")
```

□ 딕셔너리 안에 키의 존재 확인

(1) in 키워드 사용

```
# 딕셔너리 내부의 키 존재 확인
dictionary = {
    "country": "독일",
    "region": "유럽",
    "economic_level": "선진국",
    "export_item": ["자동차", "기계", "부품"]
}
```

```
# in 키워드
key = "population"
if key in dictionary:
    print(dictionary[key])
else:
    print("dictionary에 없는 키입니다")

dictionary에 없는 키입니다
```

(2) get( ) 함수

o 딕셔너리의 키로 값을 추출하는 기능을 수행. **존재하지 않는 키에 접근하면 KeyError 발생 없이 None을 출력**

```
dictionary = {
    "country": "독일",
    "region": "유럽",
    "economic_level": "선진국",
    "export_item": ["자동차", "기계", "부품"]
}
```

```
# 딕셔너리에 없는 키에 접근 시도
Bank = dictionary.get("없는 키")
print("값:", Bank)
```

```
값: None
```

```
# None으로 출력되는지 확인
if Bank == None:
    print("없는 키에 접근했어요!")
```

```
없는 키에 접근했어요!
```

3.7.4 범위 자료형과 반복문

□ 범위(range)는 for 반복문과 함께 빈번히 사용되는 자료형

o 정수로 이루어진 범위를 생성할 때 **range( ) 함수**를 사용

(1) **매개변수에 숫자 1개를 넣음: range(N)**. 여기서 N은 숫자

– 범위는 **0부터 N-1 까지** 생성됨

```
a = range(10)
list(range(10))
list(a)
```

(2) **매개변수에 숫자 2개를 넣음: range(N, M)**. 여기서 N, M은 숫자

– 범위는 **N부터 M-1 까지** 생성됨. range(start, stop)

```
a = range(1, 10)
list(range(1, 10))
list(a)
```

(3) **매개변수에 숫자 3개를 넣음 range(N, M, L).** 여기서 N, M, L은 숫자

– 범위는 **N부터 M-1 까지** 생성되고, 생성되는 숫자의 **간격은 L만큼** 차이

```
a = range(1, 10, 2)
list(range(1, 10, 2))
```

□ 위의 사례에서 10을 반드시 포함해야 함을 강조할 때는

```
a = range(1, 10 + 1)
list(a)
```

□ **TypeError**가 발생하는 이유

– range( ) 함수에는 반드시 '**정수**'를 입력해야 하는데 정수가 아닌 숫자가 들어가기 때문임

```
n = 10
a = range(0, n/3)

n = 10
a = range(0, int(n/3))      # int( ) 함수를 사용해 강제로 정수형으로 변환

n = 10
a = range(0, n//3)          # 나누기 연산자(//)를 사용해 정수로 변환. Pythonic!
```

5. while 반복문

□ 어떤 조건이 만족하는 동안 명령 블록을 수행하고, 해당 조건이 거짓이면 반복 명령문을 중단하는 구문

o **while 반복문의 기본 형태**: if 조건문과 반복문이 결합한 형태

```
while Boolean expression:
        statement
```

□ **while 반복문 예시**: for 반복문과 비슷하게 사용

①

```
i = 1                           # i 변수에 1을 할당
while i < 10:                   # i 가 10 미만인지 판단
    print(i)                    # 조건을 만족하면 i 출력
    i += 1                      # i에 1을 더하는 것을 반복하다가 i가 10 이 되면 반복 종료
```

②

```
i = 0
while i < 5:
    print("{}번째 반복.".format(i))
    i += 1
```

```
0번째 반복.
1번째 반복.
2번째 반복.
3번째 반복.
4번째 반복.
```

□ while 문과 for 문의 차이

o **while 문과 for 문**은 기본적으로 유사하고, 서로 변환이 가능. 그러나,

o **for 문**은 반복 횟수를 정확히 알고 있고, 반복 횟수가 변하지 않을 때 사용
while 반복문은 반복 횟수가 정확하지 않고, 어떤 조건을 만족하면 프로그램을 종료하고자 할 때 사용

```
# 1) for 문
for i in range(0, 5):
    print(i)

# 2) while 문
i = 0
while i < 5:
    print(i)
    i += 1
```

Q) 다음 두 코드 블록은 최종 결과값이 다른데, 어떤 것 때문에 그런 차이가 생길까?

1)

```
i = 0          # 초기값 가정
while i < 5:
    print(i)
    i += 1
```

2)

```
i = 0          # 초기값 가정
while i < 5:
    i += 1
print(i)
```

(hint)

계산 과정과 계산 결과 차이

□ 반복문의 제어

o 중간에 반복을 종료하거나 **실행 중인 반복문을 건너뛰는 경우**에 사용

o 반복문 내부에서만 사용가능한 키워드: **break, continue, else**

(1) break 키워드

o 반복문에서 논리적으로 반복을 종료

o break 키워드가 반복문에 있으면 **반복문이 강제로 종료됨**

```
# break 키워드
for i in range(10):
    if i == 5: break                # i가 5가 되면 종료
    print(i)
print("End of Program")              # 반복 종료 후 'End of Program' 출력
```

```
0
1
2
```

```
3
4
End of Program
```

(2) continue 키워드

o break 키워드와 달리 **특정 조건에서 남은 명령을 건너뛰고 다음 반복문을 수행**

```
# continue 키워드
for i in range(10):
    if i == 5: continue          # i가 5가 되면 i를 출력하지 않음
    print(i)
print("End of Program")           # 반복 종료 후 'End of Program' 출력
```

```
0
1
2
3
4
6
7
8
9
End of Program
```

(3) **else 키워드**

o 어떤 조건이 완전히 끝났을 때 한번 더 실행해주는 기능을 함

```
# else 키워드
for i in range(10):
    print(i)
else:
    print("End of Program")          # 반복 종료 후 'End of Program' 출력
```

```
0
1
2
3
4
5
6
7
8
9
End of Program
```

o for 문에 1부터 9까지 출력하라는 명령이 있다면, 이를 모두 만족한 후에 else문을 수행

- else 문이 없어도 해당 반복문이 전부 수행되면 반복문을 빠져나와 다음 코드를 수행하므로 불필요할 수도 있음
- 그러나, break 등으로 코드가 종료된다면 else 문이 수행되지 않으므로, 해당 반복문이 중간에 종료되지 않고 완벽히 수행되었는지를 확인하기 위해 사용